

Inventory Management System

Project & Development Guidelines

Technical Architecture & Core Team

June 22, 2026

Abstract

This document outlines the architectural standards, naming conventions, project structure, code styles, database setup, and API design rules for the Inventory Management System. Adherence to these guidelines is mandatory for all development tasks to ensure maintainability, scalability, and robust performance across the codebase.

Contents

1	Naming Conventions	2
1.1	C# Classes & Interfaces	2
1.2	ASP.NET Core Layers	2
1.3	Database Schema Elements	2
1.4	API Routes & Action Methods	2
2	Project Structure & Directory Layout	4
2.1	Layer Responsibilities	4
2.2	Dependency Flow	4
3	Code Style & Standards	5
3.1	General Syntax	5
3.2	Asynchronous Programming (Async/Await)	5
3.3	Error Handling	5
3.4	Nullability	6
4	Git Workflow & Commits	7
4.1	Branching Strategy	7
4.2	Conventional Commits Format	7
5	Database & Entity Framework Core Conventions	8
5.1	Fluid Configuration Setup	8
5.2	Timestamps	8
5.3	Queries	8
5.4	Database Migrations	8
6	API Design & Pagination	9
6.1	API Route Definitions	9
6.2	API Response Codes	9
6.3	Query Parameters & Pagination Model	9
6.4	JSON Serializing Properties	10

1 Naming Conventions

1.1 C# Classes & Interfaces

All C# types must strictly adhere to the following naming styles:

Element	Convention	Example
Class	PascalCase	ProductService, CategoryRepository
Interface	I prefix + PascalCase	IProductService, ICategoryRepository
Abstract class	PascalCase (no prefix)	BaseRepository
Static class	PascalCase	SlugHelper
Enum	PascalCase	UserRole

1.2 ASP.NET Core Layers

To maintain architectural separation, suffix type names with their layer-specific identifiers:

Type	Suffix	Example
Controller	Controller	ProductsController
Service Interface	I prefix + Service suffix	IProductService
Service Implementation	Service suffix	ProductService
Repository Interface	I prefix + Repository suffix	IProductRepository
Repository Implementation	Repository suffix	ProductRepository
Request DTO	Request suffix	CreateProductRequest
Response DTO	Response suffix	ProductResponse
DbContext	DbContext suffix	AppDbContext
Middleware	Middleware suffix	RoleMiddleware

1.3 Database Schema Elements

The database layer maps directly to a relational structure. To maintain cross-platform compliance, use the snake_case convention:

Element	Convention	Example
Table	snake_case (plural)	product_masuk, product_keluar
Column	snake_case	category_id, remember_token
Primary Key	id (always lowercase)	id
Foreign Key	{table}_id	product_id, supplier_id
Pivot Tables	singular_singular	product_category

1.4 API Routes & Action Methods

- **Routes:** Always use kebab-case (all lowercase with hyphens) and plural nouns for resources (e.g., /api/products-out).
- **Methods:** Use PascalCase for all public methods.

- **Async:** Always append the `Async` suffix to any asynchronous method.
- **Verb-Noun Pair:** Build descriptive action names (e.g., `GetByIdAsync`, `CreateAsync`, `DeleteAsync`).

2 Project Structure & Directory Layout

All code files must be placed within their respective logical layers inside the primary API project directory (`src/InventoryManagementSystem.Api/`):

- `Controllers/` — Thin routing and response configuration. Maps HTTP requests to services.
- `Models/` — Pure EF Core entity classes (POCOs).
- `Data/` — `AppDbContext` definition and EF Core Database Migrations.
- `DTOS/` — Input validation and API payloads:
 - `Requests/` — Data Transfer Objects for client payloads (`CreateProductRequest`).
 - `Responses/` — Data shapes returned to clients (`ProductResponse`).
- `Repositories/` — Direct DB-layer interactions containing LINQ queries and database pagination.
- `Services/` — Core business, rule execution, validation, and stock mutation math.
- `Helpers/` — Static utilities and validation adapters.
- `wwwroot/upload/products/` — Local storage directory for uploaded product asset images.
- `Program.cs` — Application container entry point and Dependency Injection (DI) service configuration.

2.1 Layer Responsibilities

Each layer is designed to handle singular architectural concerns. Crossing boundaries is prohibited.

Layer	What it does	What it doesn't do
Controller	Directs HTTP verbs to services, formats returns	Business logic execution, direct <code>DbContext</code> access
Service	Stock math, state checks, calls repositories	Direct HTTP context, raw SQL execution
Repository	LINQ optimization, <code>Where/Skip/Take</code>	Business rules, validation, HTTP formatting
Model	Defines schemas, properties, and relationship setups	Business rules or operations
DTO	Shapes client inputs/outputs with validation attributes	No Entity Framework navigation or circular bindings

2.2 Dependency Flow

`Controller` → `Service (Interface)` → `Repository (Interface)` → `DbContext`

- Controllers depend exclusively on **Service Interfaces**.
- Services depend exclusively on **Repository Interfaces**.
- Repositories depend on **DbContext**.
- Circular dependencies are strictly forbidden.

3 Code Style & Standards

3.1 General Syntax

- **Type Inference:** Use `var` only when the type is clearly evident on the right side of the expression.
- **Cleanliness:** Use expression-bodied members for simple, single-line executions.
- **Length Limitation:** Keep all methods under roughly 20 lines. Split complex methods into smaller private helper methods.
- **No Regions:** The use of `#region` and `#endregion` is prohibited. Use separate files or classes to split responsibilities instead.
- **Namespaces:** Always declare namespaces using file-scoped syntax:

```
namespace InventoryManagementSystem.Services; // File-scoped namespace
```

- **Dependency Injection:** Prefer C# 12+ primary constructor syntax for simple injection chains.

3.2 Asynchronous Programming (Async/Await)

- Ensure async coverage all the way up. Never block threads using `.Result` or `.Wait()`.
- Always pass a `CancellationToken` to async EF Core functions (e.g., `ToListAsync(ct)`, `SaveChangesAsync(ct)`).
- Prefer returning a `Task` or `Task<T>` over `async void` (excluding framework UI event hooks).

```
// Good Example
public async Task<ProductResponse?> GetByIdAsync(int id, CancellationToken ct)
{
    return await _repository.GetByIdAsync(id, ct);
}

// Bad Example
public ProductResponse? GetById(int id)
{
    return _repository.GetByIdAsync(id).Result; // Thread Blocking!
}
```

Listing 1: Asynchronous Code Best Practices Example

3.3 Error Handling

- Avoid `try-catch` blocks in standard controller layers. Use the global exception handler middleware to handle uncaught system errors.
- Services must explicitly throw domain-specific exceptions (e.g., `KeyNotFoundException`, `InvalidOperationException`).
- Allow lower-level EF Core database exceptions to bubble up automatically.

3.4 Nullability

- Nullable reference types must remain enabled in the project configurations.
- Define non-nullable strings as `string.Empty` to avoid warning prompts; use `string?` for truly optional database elements.
- Set standard navigation properties to null-forgiving default value (`null!`) when mapped by EF:

```
public Category Category { get; set; } = null!;
```

4 Git Workflow & Commits

4.1 Branching Strategy

Development branches are kept clean, structured, and short-lived:

- `main` — Production-ready branch. Strictly protected from direct pushes.
- `feature/*` — Dedicated branch for implementing new requirements (e.g., `feature/product-image-upload`).
- `fix/*` — Critical or standard bug corrections (e.g., `fix/negative-stock-guard`).
- `chore/*` — Maintenance work, project dependencies, or config changes (e.g., `chore/add-guidelines`).

To keep history clear, branches must be branched off `main`, merged back via a squash merge, and deleted immediately upon merge completion.

4.2 Conventional Commits Format

Commit messages must follow the structure below:

```
<type>(<scope>): <imperative description>
[optional body]
```

Type	Usage Scenarios
feat	Introduction of new user features.
fix	Patches, corrections, and general error-resolution code.
refactor	Code optimization changes which do not affect external behavior.
chore	Changes to builds, tooling pipelines, config settings, and updates.
docs	Any documentation-only adjustments (Wiki files, project guidelines).
style	Formatting, alignment adjustments, missing semi-colons, or white space.
test	Creating new or updating existing tests to improve quality coverage.

Examples:

- `feat(products): add image upload on create`
- `fix(products-out): prevent negative stock on outbound`
- `refactor(controllers): extract validation to service layer`

5 Database & Entity Framework Core Conventions

5.1 Fluid Configuration Setup

- All entity mappings and configurations must be configured in `AppDbContext.OnModelCreating()` using the Fluent API.
- Keep entity models (POCOs) clean. Do not use UI/Validation Data Annotations in database entity classes.
- Use explicit SQL types via `HasColumnType` when system defaults do not map to the required target scale (e.g., decimals).

```
modelBuilder.Entity<Product>(e =>
{
    e.Property(p => p.Nama).HasMaxLength(255);
    e.HasOne(p => p.Category)
        .WithMany(c => c.Products)
        .HasForeignKey(p => p.CategoryId)
        .OnDelete(DeleteBehavior.Cascade);
});
```

Listing 2: Database Mapping Configuration Example

5.2 Timestamps

- All database entities must declare `CreatedAt` and `UpdatedAt` columns.
- Track these changes inside the Service layer during mutations, not through automated overrides in the `DbContext`.
- Keep datetimes consistent by using UTC standards: `DateTime.UtcNow`.

5.3 Queries

- Database calls must always utilize their async equivalents (`ToListAsync()`, `CountAsync()`, `FirstOrDefaultAsync()`).
- Use `Include()` for any eager loading requirements. Lazy loading behavior is strictly disabled in application pipelines.
- Keep queries contained inside the Repository implementation layers. Do not write LINQ queries inside Controllers or Services.
- Use `AsNoTracking()` to optimize and speed up read-only database selections.

5.4 Database Migrations

- Build a migration using the terminal command: `dotnet ef migrations add <Name>`.
- Apply migrations to the database using: `dotnet ef database update`.
- Name migrations using descriptive PascalCase (e.g., `AddProductTable`, `AddCategoryForeignKey`).
- Do not manually modify generated migration classes. To fix schema issues, discard the current migrations or write a new one.

6 API Design & Pagination

6.1 API Route Definitions

To maintain REST architecture styles, format routes using nouns matching standard resource templates:

- GET /api/categories (Return lists)
- POST /api/categories (Write/Create resources)
- GET /api/categories/{id} (Retrieve a unique record)
- PUT /api/categories/{id} (Update record)
- DELETE /api/categories/{id} (Remove target data)

6.2 API Response Codes

Ensure responses match status descriptions:

HTTP Status	Standard Scenario Usage
200 OK	Success return payload (GET, PUT)
201 Created	Successfully created resource state (POST)
204 No Content	Successful removal operations (DELETE)
400 Bad Request	Invalid payload format or validation failures
401 Unauthorized	Invalid or expired Bearer Token
403 Forbidden	Current user lacks the required role clearances
404 Not Found	Record does not exist in the database

6.3 Query Parameters & Pagination Model

All list endpoints must accept and apply standard filtering options:

Query Param	Default	Implementation Details
page	1	Target Page Number (1-indexed sequence).
perPage	10	Records per return frame (hard cap at 100 max).
search	null	Standard text matching queries.

Responses returning paginated lists must match the following payload schema:

```
{
  "page": 1,
  "per_page": 10,
  "total": 47,
  "last_page": 5,
  "data": [
    {
      "id": 1,
      "name": "Standard Product Name"
    }
  ]
}
```

Listing 3: Standard Paginated Response Shape

6.4 JSON Serializing Properties

- Map API models to snake_case naming standards.
- Use explicit `JsonPropertyName` decorations on all API Request/Response DTO data models:

```
using System.Text.Json.Serialization;

public class CreateProductRequest
{
    [JsonPropertyName("category_id")]
    public int CategoryId { get; set; }
}
```

- **Authorization Header:** Set standard headers to authenticate endpoints: `Authorization: Bearer <token>`. All routes (except `/api/users/login`) require standard token validation.